

# SPECTRAL SCORING WITH ATTENTION-GUIDED PROPAGATION FOR AGENTIC FAILURE ATTRIBUTION

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Failure attribution for LLM agents aims to identify the decisive step responsible for task failure, which is critical for debugging and improving agentic systems. Existing methods rely on costly LLM prompting, step-level error annotations, or verified successful trajectories that lack failure signals for assistance. In this paper, we study failure attribution from *step-unlabeled failure trajectories*, where each trajectory is known to have failed but its decisive step is unknown. We propose SOAP (Spectral scOring with Attention-guided Propagation), a novel framework requiring neither step-level training labels nor a separate success-only reference set. SOAP first constructs a spectral reference space from pooled step representations and assigns each step a base error score according to its projection behavior. Importantly, later steps may exhibit stronger failure signals because they inherit an earlier mistake, causing independent scorers to favor downstream consequences over the original error. To address this *downstream contamination*, SOAP derives soft step dependencies from attention patterns and propagates downstream error evidence toward its likely upstream sources. Extensive experiments show that SOAP consistently improves attribution performance over competitive baselines.

## 1 INTRODUCTION

When an LLM-based agent fails on a long-horizon task, identifying *where the failure began* is essential for debugging, auditing, and improving the agent. An agent trajectory may contain dozens or even hundreds of reasoning, communication, and tool-use steps, while only one early decision introduces the error that ultimately causes task failure. Manually tracing such trajectories is prohibitively expensive and prone to inconsistency, especially as agents are increasingly deployed for software engineering, scientific discovery, and other complex tasks (Yao et al., 2023; Wang et al., 2024). This motivates the problem of *failure attribution*: given a failed trajectory, identify the decisive step that introduced the failure (Zhang et al., 2025c).

Failure attribution is fundamentally challenging. Existing methods typically rely either on prompting powerful LLM judges, which incurs substantial inference cost and can be unreliable over long trajectories (Zhang et al., 2025c; Banerjee et al., 2025; Wang et al., 2026; Rafi et al., 2026; Yu et al., 2025), or on post-training attribution models using failure trajectories with step-level annotations (Zhang et al., 2025a;b). Such annotations are costly and often ambiguous, since identifying the decisive step may require substantial domain expertise. OAT (Yeh et al., 2026) alleviates this burden by learning the latent dynamics of successful trajectories and detecting deviations from the learned flow. However, it has no access to failure trajectories during training, and therefore *models only what successful behavior looks like*. Moreover, its success-only reference set must be verified and may become stale as agents and task distributions evolve. This motivates a complementary question:

### **Can failed trajectories themselves enable failure attribution without step-level annotations?**

Failed trajectories provide a complementary source of supervision. In many agentic systems, whether a task has failed can be determined from execution outcomes or task evaluators, while identifying the step responsible for that failure remains costly. We therefore consider a collection of *step-unlabeled failure trajectories*, i.e., every trajectory is known to have failed, but no step is annotated as decisive or non-decisive. Each trajectory consequently contains an unlabeled mixture of ordinary steps and the decisive error. Unlike OAT (Yeh et al., 2026), such a failure-conditioned dataset directly exposes the behaviors that arise when agents fail. Yet harnessing this information

is non-trivial: without step-level membership labels, the learner must extract a meaningful failure signal without knowing which step introduced the failure. The first challenge is therefore to *identify decisive-error evidence from a step-level unlabeled mixture within failed trajectories*.

A second challenge is that the step exhibiting the strongest failure signal need not be the step that introduced the failure. Agent trajectories are autoregressive because each step is generated from preceding observations and actions. Once an error occurs, later steps operate on a corrupted context and may produce failed tool calls, contradictory reasoning, or repeated recovery attempts. These downstream consequences can be more conspicuous than the original error. For example, a plausible but incorrect tool argument may trigger a sequence of visible error messages and unsuccessful retries, causing a conventional per-step scorer to select the final retry rather than the initial incorrect action. We refer to this phenomenon as *downstream contamination* (Figure 1). Recent methods reconstruct dependencies through LLM prompting or specialized training, then perform graph search or backtracking (Rafi et al., 2026; Wang et al., 2026; Zhang et al., 2025b), which introduce substantial inference or training overhead. This motivates a lightweight alternative that extracts soft dependencies from the model to separate root causes from downstream effects.

In this paper, we propose SOAP (Spectral scOring with Attention-guided Propagation), a framework for decisive-error localization from step-unlabeled failure trajectories. At a high level, SOAP consists of two components. **First**, it constructs a *spectral reference space* from step representations extracted from the failure trajectories. Each step receives a base error score according to its projection onto a selected band of singular directions. This produces a step-level signal without requiring decisive-error annotations or a separate collection of verified successful trajectories. **Second**, SOAP addresses downstream contamination through *dependency-aware rescoring*. We derive a soft dependency graph from models’ attention patterns, where each edge approximates how strongly a later step draws on an earlier one. Rather than treating downstream failure signals as independent and competing evidence, SOAP propagates them backward toward the preceding steps on which they depend. An earlier step is therefore promoted when anomalous downstream behavior is systematically built upon it.

We evaluate SOAP on [TODO: benchmarks] using [TODO: agent systems and proxy models]. Across [TODO: number] evaluation settings, SOAP consistently improves decisive-step localization over different competitive baselines. In particular, SOAP improves [TODO: primary metric] by [TODO: result] over the strongest baseline on [TODO: benchmark]. Ablation studies further show that both spectral step scoring and attention-guided rescoring contribute to the improvements, which cannot be explained by simply favoring earlier steps or uniformly aggregating downstream scores. [TODO: see my comments in the exp section.] Our main contributions are summarized as follows:

- We formulate decisive-error localization from *step-unlabeled trajectories* within failed trajectories. This setting exploits trajectory-level failure signals while requiring neither decisive-error annotations nor a carefully verified successful trajectories.
- We identify *downstream contamination* as a central obstacle to decisive-error localization, i.e., later steps may exhibit stronger failure signals because they inherit an earlier mistake, causing independent per-step scorers to systematically localize failures too late.
- We propose SOAP, a novel framework performing spectral step scoring with attention-derived dependency propagation. Extensive experiments and ablations on [TODO: benchmarks] demonstrate when and why SOAP improves attribution. The results provide insights into leveraging step-unlabeled failure trajectories for agentic failure attribution.

Task: search for martial-arts classes within a five-minute walk of the New York Exchange, 7–9 PM

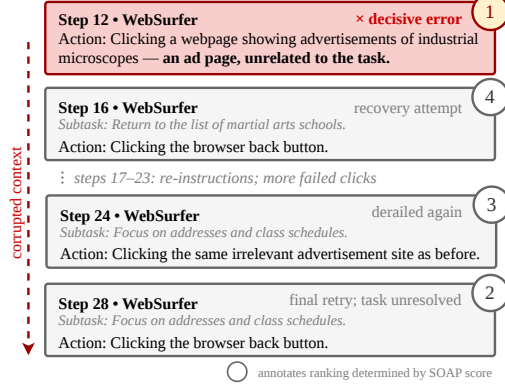


Figure 1: *Downstream contamination* in a failed Who&When trajectory. An early wrong click (step 12) lands the browser on an irrelevant page; later steps are spent on recovery attempts that derail again.

## 2 PROBLEM SETUP

We formalize the task of localizing the decisive error in a failed agent trajectory.

**Agent trajectory.** Let  $x$  denote a task description and let  $\tau = (s_1, \dots, s_T)$  be the resulting trajectory of a multi-agent system. Each step  $s_t$  contains the action or message produced at turn  $t$  and is associated with an agent  $a_t \in \{1, \dots, K\}$ . For evaluation, a failed trajectory is paired with a gold decisive-error index  $t^* \in \{1, \dots, T\}$ , corresponding to the earliest step at which the error responsible for the eventual task failure is introduced. The responsible agent is therefore  $a_{t^*}$ .

Our framework extracts step representations and attention patterns using a frozen language model  $\mathcal{M}$ . When the model that generates the trajectory exposes its internal states,  $\mathcal{M}$  can be the generating model itself. Otherwise,  $\mathcal{M}$  can be a separate open-weight proxy model, allowing the framework to analyze trajectories produced by proprietary agents whose internals are inaccessible. In either case,  $\mathcal{M}$  is used without fine-tuning on failure-attribution data.

**Failure attribution.** Given the task description  $x$  and a failed trajectory  $\tau$ , the goal of failure attribution is to estimate the decisive-error step (Zhang et al., 2025c). The learner predicts an index  $\hat{t}$  as an estimator of  $t^*$ , with the corresponding responsible-agent prediction  $\hat{a} = a_{\hat{t}}$ .

Rather than predicting the decisive-error index directly, we assign each step a scalar attribution score  $\tilde{S}(s_t) \in \mathbb{R}$ , where a larger value indicates that the step is more likely to have introduced the failure. The top-1 prediction is  $\hat{t} = \arg \max_{t \in \{1, \dots, T\}} \tilde{S}(s_t)$ . Sorting the scores further produces a ranked list of candidate steps for failure triage. We evaluate both top-1 localization and ranked-set performance in §4.

**Step-unlabeled failure trajectories.** In addition to the test trajectory, we assume access to a reference collection of failed trajectories  $\mathcal{D}_{\text{fail}} = \{\tau^{(n)}\}_{n=1}^N$ ,  $\tau^{(n)} = (s_1^{(n)}, \dots, s_{T_n}^{(n)})$ . The task-level outcome of each trajectory is known: every trajectory in  $\mathcal{D}_{\text{fail}}$  ends in failure. However, no step-level annotation is available to identify which step introduced the failure or which agent was responsible. We therefore model the pooled steps in  $\mathcal{D}_{\text{fail}}$  as an unlabeled contamination mixture (Huber, 1964)

$$P_{\text{fail}} = (1 - \pi)P_{\text{ord}} + \pi P_{\text{err}}, \quad (1)$$

where  $P_{\text{ord}}$  denotes the marginal distribution of ordinary steps,  $P_{\text{err}}$  denotes that of decisive-error steps, and  $\pi \in (0, 1)$  is an unknown contamination proportion. Equivalently, each trajectory has an unobserved error set  $\mathcal{E}^{(n)} \subseteq \{1, \dots, T_n\}$ , typically containing a single decisive step, such that  $s_t^{(n)} \sim P_{\text{err}}$  when  $t \in \mathcal{E}^{(n)}$  and  $s_t^{(n)} \sim P_{\text{ord}}$  otherwise. Eqn. 1 characterizes the marginal composition of the pooled steps; it does not assume that steps within a trajectory are temporally independent.

Our method uses  $\mathcal{D}_{\text{fail}}$  only to construct the reference representation space, without observing  $\mathcal{E}^{(n)}$  or any responsible-agent annotations. Gold step-level labels are used only for *evaluation* and, when applicable, *hyperparameter selection* on a disjoint validation split.

## 3 SOAP: SPECTRAL SCORING WITH ATTENTION-GUIDED PROPAGATION

In this section, we introduce SOAP, a framework for localizing the decisive error from step-unlabeled failure trajectories. At a high level, SOAP consists of two components. *First*, we construct a spectral reference space from the step representations in  $\mathcal{D}_{\text{fail}}$  and derive a base error score for each step (§3.1). *Second*, we estimate soft dependencies between steps from the attention patterns of the model and propagate downstream error evidence toward the earlier steps on which it depends (§3.2). The resulting score  $\tilde{S}(s_t)$  estimates how likely step  $s_t$  is to have introduced the failure.

### 3.1 SPECTRAL STEP SCORING FROM UNLABELED FAILURE TRAJECTORIES

The reference collection  $\mathcal{D}_{\text{fail}}$  contains many ordinary steps but only a small number of decisive errors within each trajectory. Consequently, the representations from LLM  $\mathcal{M}$  are dominated by recurring patterns of ordinary agent behavior, while decisive-error steps form a sparse, unlabeled component. Our key idea is to characterize this representation space through spectral decomposition and score each step according to its alignment with selected spectral directions.

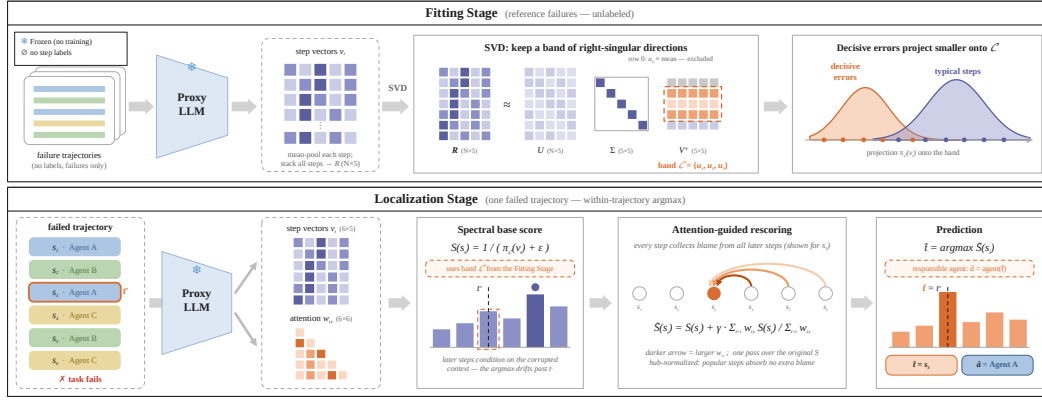


Figure 2: **Overview of SOAP.** During fitting, a frozen proxy model  $\mathcal{M}$  encodes every step of the unlabeled reference failures  $\mathcal{D}_{\text{fail}}$ , and the SVD of the stacked representations yields a spectral band  $\mathcal{C}$  capturing the common mode of ordinary steps, onto which decisive errors project weakly. During localization, the same proxy encodes the failed trajectory: each step receives the spectral base score  $S(s_t) = 1/(\pi_{\mathcal{C}}(v_t) + \epsilon)$ , and attention-derived dependency weights  $w_{i,t}$  propagate downstream error evidence back to the predecessors it builds on, moving the argmax of the rescored  $\tilde{S}$  from a late symptom onto the decisive error  $t^*$ ; the responsible agent follows as  $\hat{a} = a_{t^*}$ . SOAP involves no step labels, fine-tuning, or LLM prompting.

For a trajectory  $\tau = (s_1, \dots, s_T)$ , we encode each step in the context in which it was produced. Specifically, for step  $s_t$ , we provide the model  $\mathcal{M}$  with the task description and the trajectory context  $z_t = [x; s_1; \dots; s_t]$ , and then extract its  $d$ -dimensional step representation  $v_t$ . [refer details, e.g., context cutoff, in the appendix. —FINN]

Figure 2 summarizes the method.

**Spectral reference space.** We apply the same representation extraction procedure to every step in the reference collection  $\mathcal{D}_{\text{fail}}$ . Let  $M = \sum_{n=1}^N T_n$  be the total number of reference steps. We stack their representations into  $R = [(v_t^{(n)})^\top]_{n \in [N], t \in [T_n]} \in \mathbb{R}^{M \times d}$  where  $v_t^{(n)}$  is representation of the  $t$ -th step in the  $n$ -th sequence or trajectory. We then compute its singular value decomposition

$$R = U \Sigma V^\top. \quad (2)$$

The columns of  $V$  describe orthogonal directions of variation among the unlabeled steps, ordered by decreasing singular value.

Importantly, the reference set  $R$  is dominated by ordinary steps because each failure trajectory typically contains *only a small number of decisive errors*. Consequently, the leading singular directions may primarily encode frequent shared structure, such as response style, trajectory format, and common tool-use patterns. Conversely, components at the extreme tail may contain irrelevant variation. We therefore do not assume that the most informative signal must lie in the leading components. Instead, we consider a contiguous spectral band to represent the decisive-error information  $\mathcal{C} = \{c_{\text{begin}}, \dots, c_{\text{end}} - 1\}$ , whose boundaries are selected as described in §4.

**Spectral base score.** For a step representation  $v_t$ , we measure its average squared projection onto the selected spectral band:

$$\pi_{\mathcal{C}}(v_t) = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \langle v_t, V_{:,c} \rangle^2. \quad (3)$$

Because ordinary steps dominate the unlabeled reference corpus, decisive-error steps exhibit smaller projection magnitudes on the selected directions than ordinary steps (Figure 3). We therefore define  $S(s_t) = \frac{1}{\pi_{\mathcal{C}}(v_t) + \epsilon}$ , so that a larger score indicates stronger decisive-error evidence.  $\epsilon > 0$  is a small constant that ensures numerical stability. Note that  $R$  is constructed solely from the step-unlabeled reference collection where *no decisive-error or responsible-agent annotation is used*.

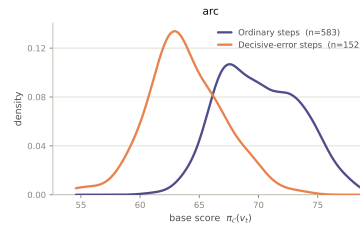


Figure 3: Base-score distribution on a CORRECT-ERROR test split. Decisive-error steps concentrate at smaller projection magnitudes than ordinary steps.

### 3.2 DEPENDENCY-AWARE RESCORING

**Attention-guided step dependencies.** [add context cutoff explanation —FINN]appendix The base score in Eqn. 3 evaluates each step independently and therefore cannot distinguish an error source from its downstream consequences. Once a decisive error occurs, later steps may inherit the corrupted context and exhibit stronger failure signals than the original error. An earlier step is therefore more plausible as the root cause when subsequent anomalous steps depend strongly on it. To capture this structure, we construct a soft dependency graph over the target trajectory.

For every pair  $(i, t)$  with  $i < t$ , we define a non-negative coefficient  $w_{i,t}$  that measures how strongly the later step  $s_t$  depends on the earlier step  $s_i$ . We derive this coefficient from the post-softmax attention produced by  $\mathcal{M}$ . Specifically, we aggregate the attention routed from tokens in  $s_t$  to tokens in  $s_i$ , yielding an attention mass  $m_{i,t}$ . We then normalize over all predecessor steps:

$$w_{i,t} = \frac{m_{i,t}}{\sum_{j<t} m_{j,t}}, \quad i < t. \quad (4)$$

Thus,  $\sum_{i<t} w_{i,t} = 1$ , and  $w_{i,t}$  represents the relative dependency of step  $s_t$  on predecessor  $s_i$ . The resulting weights provide soft evidence about which preceding steps contribute to the proxy model’s representation of a later step. They require neither repeated LLM prompting, which introduces substantial inference cost (Zhang et al., 2025c; Yeh et al., 2026), nor explicit dependency reconstruction through LLM-based graph reasoning (Rafi et al., 2026; Wang et al., 2026), nor task-specific post-training on attributed failure trajectories (Zhang et al., 2025a;b).

**Rescoring.** Given the dependency weights, we treat the error signal of a downstream step as additional evidence for the earlier steps on which it depends. For each step  $s_i$ , we compute the dependency-weighted average of the base scores of its downstream dependents:

$$B_i = \frac{\sum_{t>i} w_{i,t} S(s_t)}{\sum_{t>i} w_{i,t}}, \quad B_T = 0. \quad (5)$$

We then combine the step’s local error signal with the downstream evidence attributed to it:

$$\tilde{S}(s_i) = S(s_i) + \gamma B_i, \quad \gamma \in [0, 1]. \quad (6)$$

Here,  $\gamma$  controls the strength of dependency-aware rescoring, and  $\gamma = 0$  recovers the base score.

The normalization in Eqn. 5 produces a dependency-weighted average, which prevents an early or frequently attended step from receiving a large correction merely because it has many downstream steps. Instead, a step is promoted when the later steps that depend on it consistently exhibit strong failure signals. Conversely, an anomalous downstream step contributes little evidence to a predecessor on which it places little dependency weight.

Finally, we predict the decisive-error step and the corresponding responsible agent as

$$\hat{t} = \arg \max_{i \in \{1, \dots, T\}} \tilde{S}(s_i), \quad \hat{a} = a_{\hat{t}}. \quad (7)$$

## 4 EXPERIMENTS

### 4.1 SETUP

**Datasets.** We evaluate our method SOAP on 3 failure-attribution benchmarks comprising 5 subsets: WHO&WHEN (Zhang et al., 2025c), with algorithm-generated (WW-AG, 126 trajectories) and hand-crafted (WW-HC, 58 trajectories) subsets; CORRECT-ERROR (Yu et al., 2025) (CE, 2,226 trajectories); and TRACEELEPHANT (Chen et al., 2026), with Captain-Agent (TE-CAP, 85 trajectories) and Magentic-One (TE-MAG, 91 trajectories) subsets. They cover trajectories of varying lengths and diverse QA, coding, and tool-use tasks. Each trajectory is annotated with the decisive-error step and responsible agent. For each subset, we randomly split trajectories into unlabeled-reference, validation, and test sets using a 30/20/50% ratio. The reference annotations are hidden and used only to construct the spectral matrix  $R$ ; validation labels are used for hyperparameter selection, and test labels only for final evaluation. Splitting is performed at the trajectory level. Full statistics and preprocessing details are provided in Appendix ??.

[add number of steps,appendix —FINN]

Table 1: **Main comparison in the *without-ground-truth* setting.** Results are step-level accuracy (%), averaged over three seeds (std in Appendix ??). No method receives the gold task answer. GPT denotes GPT-4o; results with GPT-5 as the judge are in Appendix ??. **Supervised** marks methods that consume step-level error annotations. Best per column within each block in **bold**, second best underlined. [revised it —JUNLIN]

| Method                              | Supervised | WW-AG        | WW-HC        | CE           | TE-Cap       | TE-Mag       |
|-------------------------------------|------------|--------------|--------------|--------------|--------------|--------------|
| <b>GPT</b>                          |            |              |              |              |              |              |
| <i>Prompt-based methods</i>         |            |              |              |              |              |              |
| All-at-Once (Zhang et al., 2025c)   |            | 14.81        | 6.90         | 28.39        | 16.28        | 5.80         |
| Step-by-Step (Zhang et al., 2025c)  |            | 20.63        | 13.79        | 44.47        | 13.95        | 13.04        |
| Binary Search (Zhang et al., 2025c) |            | 22.22        | 4.60         | 9.70         | 9.30         | 6.52         |
| CORRECT (Yu et al., 2025)           | ✓          | 15.34        | 4.60         | 35.12        | 10.85        | <u>13.77</u> |
| CHIEF (Wang et al., 2026)           |            | 25.93        | 14.94        | 38.56        | 13.95        | 8.70         |
| RAFFLES (Zhu et al., 2026a)         |            | 35.98        | 18.39        | 53.05        | 23.26        | 23.91        |
| <b>Qwen3.5-9B</b>                   |            |              |              |              |              |              |
| <i>Prompt-based methods</i>         |            |              |              |              |              |              |
| All-at-Once (Zhang et al., 2025c)   |            | —            | —            | —            | —            | —            |
| Step-by-Step (Zhang et al., 2025c)  |            | —            | —            | —            | —            | —            |
| Binary Search (Zhang et al., 2025c) |            | —            | —            | —            | —            | —            |
| CORRECT (Yu et al., 2025)           | ✓          | —            | —            | —            | —            | —            |
| CHIEF (Wang et al., 2026)           |            | —            | —            | —            | —            | —            |
| RAFFLES (Zhu et al., 2026a)         |            | —            | —            | —            | —            | —            |
| <i>Training-based methods</i>       |            |              |              |              |              |              |
| StepFinder (Zhu et al., 2026b)      | ✓          | 15.87        | <u>13.33</u> | 30.03        | <u>18.29</u> | 6.67         |
| OAT (Yeh et al., 2026)              | ✗          | <u>16.72</u> | 10.11        | <u>56.50</u> | 15.35        | <u>18.84</u> |
| <b>SOAP (ours)</b>                  | ✗          | <b>47.62</b> | <b>34.48</b> | <b>61.78</b> | <b>35.66</b> | <b>23.19</b> |
| <b>DeepSeek-R1-Distill-Llama-8B</b> |            |              |              |              |              |              |
| <i>Prompt-based methods</i>         |            |              |              |              |              |              |
| All-at-Once (Zhang et al., 2025c)   |            | —            | —            | —            | —            | —            |
| Step-by-Step (Zhang et al., 2025c)  |            | —            | —            | —            | —            | —            |
| Binary Search (Zhang et al., 2025c) |            | —            | —            | —            | —            | —            |
| CORRECT (Yu et al., 2025)           | ✓          | —            | —            | —            | —            | —            |
| CHIEF (Wang et al., 2026)           |            | —            | —            | —            | —            | —            |
| RAFFLES (Zhu et al., 2026a)         |            | —            | —            | —            | —            | —            |
| <i>Training-based methods</i>       |            |              |              |              |              |              |
| StepFinder (Zhu et al., 2026b)      | ✓          | <u>18.94</u> | 10.80        | 36.91        | 14.73        | 4.20         |
| OAT (Yeh et al., 2026)              | ✗          | 12.70        | <u>15.86</u> | <u>52.50</u> | <u>17.98</u> | <u>13.04</u> |
| <b>SOAP (ours)</b>                  | ✗          | <b>45.50</b> | <b>28.74</b> | <b>64.68</b> | <b>42.64</b> | <b>30.43</b> |

**[Baselines.** We compare SOAP against two families of baselines. First, *prompt-based* methods include All-at-Once (), Step-by-Step (), and Binary Search (Zhang et al., 2025c), the schema-retrieval method CORRECT (Yu et al., 2025), the causal-graph method CHIEF (Wang et al., 2026), and the fault-reasoning judge RAFFLES (Zhu et al., 2026a). These methods ask a judge LLM to name the decisive step. Second, *training-based* methods include OAT (Yeh et al., 2026), which learns successful-trajectory dynamics through one-class training, and StepFinder (Zhu et al., 2026b), which trains a step classifier on synthetically corrupted trajectories with step-level error labels. —JUNLIN]

**[Implementation details.** We use Qwen3.5-9B (Qwen Team, 2025) and DeepSeek-R1-Distill-Llama-8B (Guo et al., 2025) as backbones. We obtain step representations by mean-pooling token hidden states. We select the representation layer, the spectral band  $\mathcal{C}$ , the attention layer band, and the propagation strength  $\gamma$  on the validation set (we show the stability of SOAP to each hyperparameter in Section 4.3). All training-based baselines use the same two backbones and test data as SOAP, with the hyperparameters recommended in their papers. All prompt-based baselines use GPT-4o as the judge (results with GPT-5 are in Appendix ??). Following Zhang et al. (2025c), we report *Step-Level Accuracy* (we also reports additional metrics in Appendix ??). Further implementation details are provided in Appendices ?? and N. [add appendix for agent-level and other metrics —JUNLIN]

## 4.2 MAIN RESULTS

**[Comparison with competitive baselines.** Table 1 compares SOAP against both baseline families on the five subsets, strictly without the ground-truth answer. SOAP achieves the best step-level

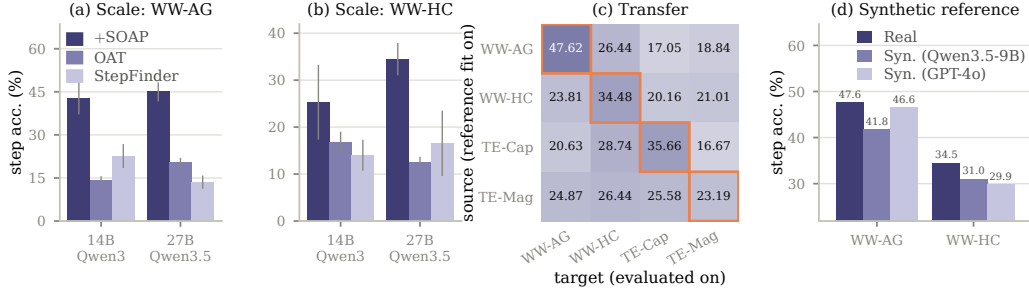


Figure 4: (a) **Scalability to larger backbones** on WW-AG. (b) **Scalability to larger backbones** on WW-HC. Step-level accuracy (%) with Qwen3-14B and Qwen3.5-27B as the backbone; error bars  $\pm 1$  std. (c) **Generalization across data distributions**: source rows, target columns; the outlined diagonal is the in-distribution result of Table 1. (d) **Results with synthetic trajectories**: the reference is constructed on generated trajectories instead of the real train split. (c) and (d) use backbone Qwen3.5-9B.

accuracy on all five subsets with both backbones, and outperforms the strongest prompt-based and training-based baselines by large margins (e.g., 47.62 vs. 35.98 for RAFFLES (Zhu et al., 2026a) and 16.72 for OAT (Yeh et al., 2026) on WW-AG). The advantage over the closed-source judges holds on both GPT-4o and GPT-5 (GPT-5 results in Appendix ??). Notably, SOAP is the only method constructed on the actual trajectories while requiring no step-level annotation. The step-unlabeled failure trajectories already carry the attribution signal, which success-only training (OAT) and synthetic supervision (StepFinder) miss. Agent-level results for the same grid appear in Appendix ??. —JUNLIN]

**[Scalability to the with-GT setting.** To assess the scalability of SOAP to the with-GT setting, we provide the gold task answer to all methods and compare them in Table 2. When provided with the ground-truth answer, SOAP still achieves the best accuracy on both subsets, and outperforms the strongest prompt-based and training-based baselines (e.g., 43.39 vs. – for RAFFLES (Zhu et al., 2026a) and 22.12 for OAT (Yeh et al., 2026) on WW-AG). Results on DeepSeek-8B appear in Appendix A. —JUNLIN]

**[Scalability to larger backbones.** To assess scalability, we evaluate SOAP on two larger backbones, Qwen3-14B and Qwen3.5-27B. As shown in Figure 4, SOAP achieves stable accuracy on both backbones and outperforms the SOTA baselines (e.g., 44.97 vs. 22.65 for StepFinder (Zhu et al., 2026b) on WW-AG). These results confirm that SOAP remains effective on larger backbones. —JUNLIN]

**[Generalization across data distributions.** We construct the reference on a source subset and directly evaluate SOAP on a different target subset, for every pair among {WW-AG, WW-HC, TE-Cap, TE-Mag}. As shown in Figure 4(c), SOAP transfers across subsets: for example, constructing on TE-Cap and testing on WW-HC achieves 28.74, compared with the in-domain result of 34.48. These results indicate that the spectral reference captures failure structure that is largely invariant to the source distribution. Results on DeepSeek-8B appear in Appendix ??. —JUNLIN]

**[Results with synthetic trajectories.** SOAP constructs the reference on unlabeled trajectories, so we further evaluate whether it requires trajectories from the target system. We generate synthetic trajectories with Qwen3.5-9B and GPT-4o, construct  $R$  on them instead of the real train split, and keep the validation and test splits of Table 1 unchanged. As shown in Figure 4(d), the synthetic references stay close to the real one: for example, the GPT-4o-generated reference achieves 46.56 on WW-AG, compared with 47.62 for the real train split. These results indicate that SOAP does not require real trajectories from the target system. Generation details are in Appendix ??. —JUNLIN]

Table 2: **Comparison in the with-ground-truth setting.** Results are step-level accuracy (%) on Who&When, with the gold task answer provided. All methods use Qwen3.5-9B as the backbone (DeepSeek-8B in Appendix A). Marks follow Table 1. [The table was revised by. —JUNLIN]

| Method                        | Sup. | WW-AG        | WW-HC        |
|-------------------------------|------|--------------|--------------|
| <i>Prompt-based methods</i>   |      |              |              |
| All-at-Once                   |      | –            | –            |
| Step-by-Step                  |      | –            | –            |
| Binary Search                 |      | –            | –            |
| CORRECT                       | ✓    | –            | –            |
| CHIEF                         |      | –            | –            |
| RAFFLES                       |      | –            | –            |
| <i>Training-based methods</i> |      |              |              |
| StepFinder                    | ✓    | 18.52        | 12.87        |
| OAT                           | ✗    | 22.12        | 8.05         |
| <b>SOAP (ours)</b>            | ✗    | <b>43.39</b> | <b>34.48</b> |

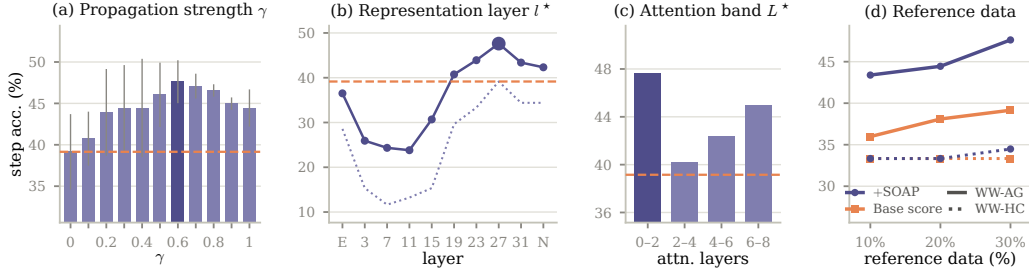


Figure 5: **Ablation studies.** (a) Sensitivity to the propagation strength  $\gamma$ , (b) sensitivity to the representation layer  $l^*$ , (c) sensitivity to the attention layer band  $L^*$ , and (d) quantity of unlabeled reference data. All results use backbone Qwen3.5-9B. Panels (a–c) vary one axis at a time from the selected configuration of Table 1 on WW-AG: the dark bar or large marker is the selected value, the dashed orange line is the base score with no rescoring ( $\gamma=0$ ), and the dotted line in (b) is the base score at each layer. Panel (d) constructs the reference on  $\{10, 20, 30\}$ % of the corpus: SOAP in purple, the base score in orange; WW-AG solid, WW-HC dotted.

Table 4: **Alternatives to attention-guided rescoring.** Step-level accuracy (%), Qwen3.5-9B; all rows share the spectral base score. Weight rows replace only the dependency weights; position rows replace the whole correction ( $\lambda$  selected per column from  $\{0.1, \dots, 1.0\}$ ). The last row equals SOAP in Table 1. Best in **bold**.

| Rescoring design                | WW-AG        | WW-HC        | CE           | TE-Cap       | TE-Mag       |
|---------------------------------|--------------|--------------|--------------|--------------|--------------|
| Base score (no rescoring)       | 39.15        | 33.33        | 61.38        | 33.33        | 21.01        |
| <i>Content-agnostic weights</i> |              |              |              |              |              |
| Uniform, unnormalized           | 16.93        | 16.09        | 57.85        | 14.73        | 5.07         |
| Uniform, normalized             | 40.21        | <b>34.48</b> | 60.78        | <b>35.66</b> | 15.94        |
| <i>Position heuristics</i>      |              |              |              |              |              |
| Temporal bias, z-scored         | 40.74        | <b>34.48</b> | 61.62        | 34.11        | 21.01        |
| Temporal bias, raw              | 22.22        | 3.45         | 41.65        | 6.20         | 5.07         |
| Earliest of top-5               | 21.16        | 14.94        | 4.51         | 9.30         | 5.07         |
| Attention-guided (ours)         | <b>47.62</b> | <b>34.48</b> | <b>61.78</b> | <b>35.66</b> | <b>23.19</b> |

### 4.3 ANALYSIS

**[Comparison with alternative scoring functions.** We compare the spectral base score with seven alternatives in Table 3: perplexity, random subspace, top subspace, tail subspace, full spectrum, and the  $\ell_1$  and  $\ell_2$  norms (details in Appendix ??). Our spectral band remains the best on both subsets, and the best alternative reaches only 18.52 on WW-AG, compared with 39.15 for ours. These results indicate that the decisive-error signal lies in a selected spectral band rather than in likelihood or vector magnitude. —JUNLIN]

**[Effect of attention-guided rescoring.** We compare the attention-guided weights of Eqn. 5 with two content-agnostic weights (uniform, normalized or unnormalized) and two position heuristics (temporal bias, z-scored or raw, and earliest of top-5) in Table 4. Attention-guided rescoring matches or outperforms every alternative on all five subsets: for example, it reaches 47.62 on WW-AG, while the normalized uniform weights reach 40.21 and the unnormalized ones collapse to 16.93. These results indicate that the gain comes from which successors the attention selects, not from aggregation or position. —JUNLIN]

**[Sensitivity to the hyperparameters.** We vary the propagation strength  $\gamma$ , the representation layer  $l^*$ , and the attention layer band  $L^*$  one at a time. As shown in Figure 5, accuracy peaks at  $\gamma = 0.6$  and stays above the base score over a wide range, the signal is strongest at late layers, and the attention band changes accuracy the least. These results suggest that SOAP is stable to each choice. WW-HC, TraceElephant, DeepSeek-8B, and the context window  $w$  of the dependency weights are in Appendix ??. —JUNLIN]

Table 3: **Comparison with alternative scoring functions.** Results are step-level accuracy (%) on backbone Qwen3.5-9B, with no rescoring.

| Scoring function     | WW-AG        | WW-HC        |
|----------------------|--------------|--------------|
| Perplexity           | 3.70         | 8.05         |
| Random subspace      | 13.23        | 9.20         |
| Top subspace         | 12.70        | <b>33.33</b> |
| Tail subspace        | 6.35         | 6.90         |
| Full spectrum        | 12.17        | 31.03        |
| Norm $\ell_1$        | 18.52        | 18.39        |
| Norm $\ell_2$        | 14.29        | 26.44        |
| Spectral band (ours) | <b>39.15</b> | <b>33.33</b> |

**[Quantity of unlabeled reference data.** We construct  $R$  on random subsamples covering  $\{10, 20, 30\}\%$  of the corpus, with the validation and test splits fixed, and re-select the configuration at each fraction. As shown in Figure 5(d), SOAP needs little reference data: with a third of the train split (twelve trajectories on WW-AG and six on WW-HC), accuracy is within a few points of the full-data result (43.39 vs. 47.62 on WW-AG). These results indicate that a handful of unlabeled trajectories suffices to construct the reference. —JUNLIN]

**Qualitative example.** Figure 6 shows a TE-Cap trajectory on which rescoring flips the prediction from wrong to right. The base score peaks late, on an Orchestrator turn that only summarizes earlier steps. The dependency weights route this late anomaly back to the decisive error, a malformed program produced by the extraction agent four steps earlier.

efficiency comparison to be in the appendix.

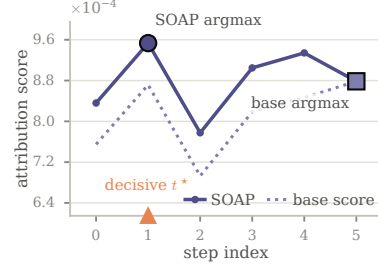


Figure 6: **Qualitative example.** Per-step scores on a TE-Cap trajectory with backbone Qwen3.5-9B. The base score (dotted) peaks late; SOAP (solid) lifts the decisive error  $t^*$  (orange marker) to the top. The large square and circle mark each score’s argmax.

## 5 RELATED WORK

**Agentic failure attribution** has gained interest recently for ensuring LLM-based agents’ reliability (Qi et al., 2026; Barke et al., 2026). Zhang et al. (2025c) formulate automated failure attribution, introduce the Who&When benchmark, and evaluate All-at-Once, Step-by-Step, and Binary Search LLM judges; complementary work categorizes common multi-agent failure modes (Cemri et al., 2026; Deshpande et al., 2025; Raj et al., 2026). Subsequent methods either improve inference-time prompting (Banerjee et al., 2025; Zhu et al., 2026a; Yu et al., 2025; Bakish et al., 2026) or train dedicated attributors. For instance, AgenTracer and StepFinder instead learn from labeled or synthetically corrupted trajectories using reinforcement learning or temporal-semantic modeling (Zhang et al., 2025a; Zhu et al., 2026b). These approaches, however, often rely on *computationally intensive inference or large-scale step-level error annotations*, limiting their practicality for real-world deployment. A recent approach OAT mitigates this by learning continuous latent dynamics from clean successful trajectories by one-class learning (Yeh et al., 2026), which *provides no decisive error signals* in their data unlike our adopted step-unlabeled failure trajectories thus underperforms our SOAP (Table ??). Several methods model cross-step dependencies (Rafi et al., 2026). GraphTracer derives information-flow graphs from references to prior outputs and learns to trace root causes and propagation paths (Zhang et al., 2025b); CHIEF construct causal dependency graphs during prompting through multi-stage LLM reasoning (Wang et al., 2026). We compare with both of them in Table ?. Liu et al. (2026) use attention signals for failure attribution but different from SOAP, they calculate attention scores on steps with high negative log-likelihood for two-stage localization.

## 6 CONCLUSION

[We propose SOAP, a novel framework for failure attribution in LLM-based multi-agent systems. Our framework scores each step against a spectral reference constructed on unlabeled failure trajectories, and routes late anomalies back to the decisive error with attention-guided propagation. SOAP is lightweight, requires no step-level annotation, and can be readily applied to off-the-shelf backbones. Extensive experiments show that SOAP consistently outperforms prompt-based and training-based baselines across benchmarks, backbones, and settings. We hope our work can inspire future research on failure attribution and more broadly on building reliable multi-agent systems. —JUNLIN]

## AI USE STATEMENT

(This section is **required** and does not count toward the page limit.)

In this work, we used generative AI tools for [tasks with required disclosure]. We have not used generative AI tools for [other tasks with required disclosure], and [the rest of the required disclosure tasks] are not applicable to this work. Additionally, we used generative AI tools for [tasks with recommended disclosure]. We have reviewed all AI-assisted work. [Elaborate. For example, “we checked LLM-generated research ideas for potential plagiarism through a manual literature survey”, “LLM-generated code was verified and tested for correctness by 2 authors”, etc.]. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

See the ICLR 2027 AI Policy for Authors for more details. This statement should not be more than 1 page.

#### ETHICS STATEMENT

(This section is **recommended** and does not count toward the page limit.)

If authors feel that their paper submission raises questions regarding the Code of Ethics, they are encouraged to include a paragraph of Ethics Statement (at the end of the main text before references) to address potential concerns where appropriate. Topics include, but are not limited to, studies that involve human subjects, practices to data set releases, potentially harmful insights, methodologies and applications, potential conflicts of interest and sponsorship, discrimination/bias/fairness concerns, privacy and security issues, legal compliance, and research integrity issues (e.g., IRB, documentation, research ethics). This statement should not be more than 1 page.

#### REPRODUCIBILITY STATEMENT

(This section is **recommended** and does not count toward the page limit.)

It is important that the work published in ICLR is reproducible. Authors are strongly encouraged to include a paragraph-long Reproducibility Statement at the end of the main text (before references) to discuss the efforts that have been made to ensure reproducibility. This paragraph should not itself describe details needed for reproducing the results, but rather reference the parts of the main paper, appendix, and supplemental materials that will help with reproducibility. For example, for novel models or algorithms, a link to an anonymous downloadable source code can be submitted as supplementary materials; for theoretical results, clear explanations of any assumptions and a complete proof of the claims can be included in the appendix; for any datasets used in the experiments, a complete description of the data processing steps can be provided in the supplementary materials. Each of the above are examples of things that can be referenced in the reproducibility statement.

#### REFERENCES

- Yarden Bakish, Amir Dudai, Roy Ganz, Oren Nuriel, Elad Ben Avraham, Mor Shpigel Nacson, and Ron Litman. Adaptive influence graphs for failure attribution in multi-agent systems. *arXiv preprint arXiv:2608.24361*, 2026.
- Adi Banerjee et al. Where did it all go wrong? a hierarchical look into multi-agent error attribution. *arXiv preprint arXiv:2510.04886*, 2025.
- Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. AgentRx: Diagnosing AI agent failures from execution trajectories. *arXiv preprint arXiv:2602.02475*, 2026.
- Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. URL <https://openreview.net/forum?id=fAjbYBmonr>.
- Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. Seeing the whole elephant: A benchmark for failure attribution in LLM-based multi-agent systems. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.),

- Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 19888–19905, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.912. URL <https://aclanthology.org/2026.acl-long.912/>.
- Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. Trail: Trace reasoning and agentic issue localization. *arXiv preprint arXiv:2505.08638*, 2025.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.
- Yang Liu, Hongjiang Feng, Junsong Pu, and Zhuangbin Chen. MASPrism: Lightweight failure attribution for multi-agent systems using prefill-stage signals. *arXiv preprint arXiv:2605.07509*, 2026.
- Yunjia Qi, Zehua Yin, Xintong Shi, Hao Peng, Songyuanyi Lu, Yixian Liu, Richeng Xuan, Yuhong Liu, Zhichao Hu, Xiaozhi Wang, Lei Hou, Bin Xu, and Juanzi Li. TRAJDEBUG: Tracing error lifecycle to identify critical failures in long-horizon agent trajectories. *arXiv preprint arXiv:2608.06346*, 2026.
- Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Md Nakhla Rafi, Md Ahasanuzzaman, Dong Jae Kim, Zhijie Wang, and Tse-Hsun Chen. Falat: Tracing failures in llm agent trajectories via dependency-guided search, 2026. URL <https://arxiv.org/abs/2606.00765>.
- Harsh Raj, Vipul Gupta, Anas Mahmoud, Razvan-Gabriel Dumitru, Darvin Yi, Aakash Sabharwal, and Yunzhong He. Model or harness? an interaction-centric taxonomy for localizing agent failures. *arXiv preprint arXiv:2607.28802*, 2026.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. doi: 10.1007/s11704-024-40231-1.
- Yawen Wang, Wenjie Wu, Junjie Wang, and Qing Wang. From flat logs to causal graphs: Hierarchical failure attribution for llm-based multi-agent systems, 2026. URL <https://arxiv.org/abs/2602.23701>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL [https://openreview.net/forum?id=WE\\_vluYUL-X](https://openreview.net/forum?id=WE_vluYUL-X).
- Samuel Yeh, Yiwen Zhu, Shaleen Deep, and Sharon Li. Tracing agentic failure from the flow of success. *arXiv preprint arXiv:2607.12747*, 2026.
- Yifan Yu, Moyan Li, Shaoyuan Xu, Jinmiao Fu, Xinhai Hou, Fan Lai, and Bryan Wang. CORRECT: COndenseD eRror RECognition via knowledge transfer in multi-agent systems. *arXiv preprint arXiv:2509.24088*, 2025.
- Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. AgenTracer: Who is inducing failure in the llm agentic systems? *arXiv preprint arXiv:2509.03312*, 2025a.
- Heng Zhang, Yuling Shi, Xiaodong Gu, Haochen You, Zijian Zhang, Lubin Gan, Yilei Yuan, and Jin Huang. Graphtracer: Graph-guided failure tracing in llm agents for robust multi-turn deep search, 2025b. URL <https://arxiv.org/abs/2510.10581>.

- Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. In *International Conference on Machine Learning (ICML)*, 2025c.
- Chenyang Zhu, Spencer Hong, Jingyu Wu, Kushal Chawla, Yuhui Tang, Youbing Yin, Nathan Wolfe, Erin Babinsky, and Daben Liu. RAFFLES: Reasoning-based attribution of faults for LLM systems. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7659–7688, Rabat, Morocco, March 2026a. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.359. URL <https://aclanthology.org/2026.eacl-long.359/>.
- Taiyu Zhu, Yifan Wu, Weilin Jin, Ying Li, and Gang Huang. StepFinder: A temporal semantic framework for failure attribution in multi-agent systems. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2026b.

## A WITH-GROUND-TRUTH BASELINES

Table 5 repeats Table 1 in the *with-ground-truth* setting, where the prompting and voting baselines are additionally given the correct answer to the task as a reference signal (Zhang et al., 2025c). CRR uses no ground-truth answer and is shown at its without-ground-truth numbers for reference; the comparison illustrates that it remains competitive with answer-aided baselines despite using strictly less information.

Table 5: Failure attribution in the *with-ground-truth* setting (baselines given the correct task answer), proxy = Qwen3-8B. CRR uses no ground-truth answer (shown at its without-GT numbers). Counterpart to Table 1.

| Method                              | Algorithm-Generated |      | Hand-Crafted |      |
|-------------------------------------|---------------------|------|--------------|------|
|                                     | Agent               | Step | Agent        | Step |
| All-at-Once (Zhang et al., 2025c)   | –                   | –    | –            | –    |
| Step-by-Step (Zhang et al., 2025c)  | –                   | –    | –            | –    |
| Binary Search (Zhang et al., 2025c) | –                   | –    | –            | –    |
| ECHO (Banerjee et al., 2025)        | –                   | –    | –            | –    |
| CORRECT (Yu et al., 2025)           | –                   | –    | –            | –    |
| CRR (no GT, for reference)          | –                   | –    | –            | –    |

**Agent-level plug-and-play.** Table 6 is the agent-level counterpart of Table ??.

Table 6: Agent-Level Accuracy with and without CRR. Agent-level counterpart of Table ??.

| Proxy        | Subset              | Norm |      | SVD  |      |
|--------------|---------------------|------|------|------|------|
|              |                     | Base | +CRR | Base | +CRR |
| Qwen3-8B     | Algorithm-Generated | –    | –    | –    | –    |
|              | Hand-Crafted        | –    | –    | –    | –    |
| Llama-3.1-8B | Algorithm-Generated | –    | –    | –    | –    |
|              | Hand-Crafted        | –    | –    | –    | –    |

## B RANKED-CANDIDATE METRICS

Table 7: Ranked-candidate metrics (step@5, MRR, agent@3). Single-verdict baselines emit one step and cannot populate a ranked set; CRR returns an ordered shortlist. Proxy = Qwen3-8B.

| Method                          | Algorithm-Generated |     |         | Hand-Crafted |     |         |
|---------------------------------|---------------------|-----|---------|--------------|-----|---------|
|                                 | step@5              | MRR | agent@3 | step@5       | MRR | agent@3 |
| Best base scorer ( $\gamma=0$ ) | –                   | –   | –       | –            | –   | –       |
| CRR                             | –                   | –   | –       | –            | –   | –       |

Table 8: Step-Level Accuracy with tolerance  $\pm 1$  and  $\pm 2$  for the main methods (hand-crafted, Qwen3-8B).

| Method                          | Exact | $\pm 1$ | $\pm 2$ |
|---------------------------------|-------|---------|---------|
| All-at-Once                     | –     | –       | –       |
| Step-by-Step                    | –     | –       | –       |
| Binary Search                   | –     | –       | –       |
| Best base scorer ( $\gamma=0$ ) | –     | –       | –       |
| CRR                             | –     | –       | –       |

Table 9: Robustness to the random split. Step-Level Accuracy under a different seed; cf. Table ??.

| Proxy    | Subset       | Norm |      | SVD  |      |
|----------|--------------|------|------|------|------|
|          |              | Base | +CRR | Base | +CRR |
| Qwen3-8B | Hand-Crafted | –    | –    | –    | –    |

## C STEP-LEVEL ACCURACY WITH TOLERANCE

## D DIFFERENT RANDOM SPLIT

## E SINGLE-PASS VS. CASCADING DISCOUNT

## F DEPENDENCY-WEIGHT SOURCE

## G CONTEXT SELECTION

## H POOLING AND LAYER SWEEP

## I COMPUTE AND EFFICIENCY

## J HYBRID WITH LLM-AS-JUDGE

## K ROBUSTNESS TO THE SVD REFERENCE

## L QUALITATIVE CASE STUDY

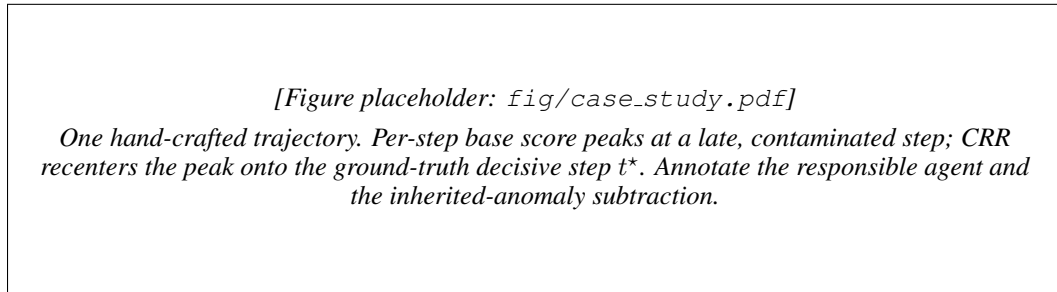


Figure 7: Case study illustrating the contamination correction on a single trajectory.

## M PROXY MODELS AND PER-MODEL RESULTS

We evaluate CRR on six proxy models spanning the Qwen and Llama families and scales from 8B to 27B. Table 17 reports the per-model breakdown of the SVD base scorer and CRR on both subsets

Table 10: Effect of reading the original  $S(s_i)$  (single-pass, default) vs. the rescored  $\tilde{S}(s_i)$  (cascading) in equation ?? . Step-Level Accuracy, hand-crafted, Qwen3-8B.

| Variant               | Algorithm-Generated | Hand-Crafted |
|-----------------------|---------------------|--------------|
| Single-pass (default) | –                   | –            |
| Cascading             | –                   | –            |

Table 11: Attention-mass weights (equation ??) vs. hidden-state-similarity weights for  $w_{i,t}$ . Step-Level Accuracy.

| Weight source           | Algorithm-Generated | Hand-Crafted |
|-------------------------|---------------------|--------------|
| Hidden-state similarity | –                   | –            |
| Attention mass (ours)   | –                   | –            |

(without ground truth). The SVD base scorer is consistently competitive across models, and CRR consistently improves it, confirming that our conclusions are not tied to a single proxy.

## N SETUP OF THE TRAINING-BASED BASELINES

This section details how we run the two training-based baselines of Table 1, OAT (Yeh et al., 2026) and StepFinder (Zhu et al., 2026b). Both methods, like SOAP, score a trajectory through the internal vectors of an open-weight model rather than by prompting a judge. We reproduce each from the code its authors released, train it on the training data its paper prescribes, run it on the same two backbones as SOAP, and score its predictions on the identical test splits under the shared evaluation protocol (§4.1). Neither method sees our reference or validation trajectories during training. We describe each method’s pipeline, its training data, and every place our setup departs from the released code.

**OAT.** OAT learns the latent dynamics of *successful* trajectories and flags the steps of a failed trajectory that depart from them. Each trajectory is serialized as one document — the task description followed by every step under a header naming its index and agent — and encoded in a single forward pass of the backbone; a step’s representation is the mean of its tokens’ last-layer hidden states, reduced to 64 dimensions by PCA fitted on the training trajectories. A neural controlled differential equation is then trained to predict each step’s latent state from the trajectory so far, with squared-error loss. Training is one-class: because our benchmarks contain only failures, OAT trains on the 103 successful tool-use trajectories released with the paper (MCP-Atlas), which is the paper’s own out-of-distribution protocol; no trajectory from our benchmarks enters training, and one trained model serves every subset. At test time, CORAL aligns the second-order statistics of each subset’s representations with those of the training set (using no labels); a step’s anomaly score is the squared distance between its predicted and observed latent state, and the predicted decisive step is the argmax. OAT predicts no agent, so the responsible-agent prediction is the agent of the predicted step. Following the released code, human and environment turns are not scored; a trajectory whose annotated step falls on such a turn is kept and counted as an error. We train five models per backbone (seeds 42–46) and report the mean. In the with-GT setting, the gold answer is appended to the task description with the same phrasing the prompt-based baselines use; the training corpus carries no answers, so the trained model is shared between the two settings. We note that the paper’s own benchmark annotates every step that contributed to a failure and reports set metrics, whereas our benchmarks annotate one decisive step, so the numbers printed in the paper are not directly comparable to Table 1.

**StepFinder.** StepFinder is a supervised step classifier. Each step is embedded twice — 128 dimensions for its text, 32 for the name of its agent — and the sequence of step embeddings passes through a BiLSTM, an agent-aware attention layer, and a scoring head with multi-scale differencing and a position prior; training minimizes a cross-entropy loss across the steps of each trajectory plus a self-supervised next-step prediction loss. Training uses the step-labeled failure corpora released with the paper, which regenerate each annotated task into roughly eighteen alternative failures: 1,564 trajectories in the CaptainAgent style and 2,604 in the Magentic-One style. Because the agent embedding is a text embedding of the agent’s name, each evaluation subset is paired with the corpus from the

Table 12: Context selection for representation and attention extraction: all predecessors vs. dependency-graph predecessors (hand-crafted only). Step-Level Accuracy.

| Context          | Base | +CRR |
|------------------|------|------|
| All predecessors | –    | –    |
| Dependency graph | –    | –    |

Table 13: Base-representation pooling ( $\phi$ ) and layer  $l^*$  for the SVD base scorer. Step-Level Accuracy, hand-crafted, Qwen3-8B.

| Pooling | early $l^*$ | mid $l^*$ | late $l^*$ | best |
|---------|-------------|-----------|------------|------|
| Mean    | –           | –         | –          | –    |
| Last    | –           | –         | –          | –    |

agent system that produced it: WW-AG and TE-Cap with the CaptainAgent corpus, WW-HC and TE-Mag with the Magentic-One corpus. CE matches neither system and falls back to the Magentic-One corpus, so its column is out-of-domain transfer. To place StepFinder on the same backbones as SOAP, we replace the paper’s encoder (Qwen3-Embedding-0.6B) with the backbone’s last-layer hidden state at the final token and keep the leading 128 (text) and 32 (agent) coordinates, exactly as the released code slices its embeddings; the slice is principled for the paper’s embedding model, whose leading coordinates are trained to carry the most information, and arbitrary for a decoder. For reference, with the paper’s own encoder StepFinder reaches 24.02 / 11.72 / 32.75 / 17.98 / 21.16 on the five subsets of Table 1 — higher than the backbone rows on WW-AG and TE-Mag, comparable elsewhere, and still far below SOAP on every subset. Our setup departs from the released code in two places, both protocol repairs. First, the released training loop selects the checkpoint by accuracy on the test set; we instead hold out a fraction of the training tasks — split by question, so the near-duplicate regenerations of one task never straddle the boundary — and select on that holdout, never touching the test data. Second, we score one trajectory at a time, so the position prior is normalized by the trajectory’s own length as in the paper’s equations rather than by the padded width of a batch. As for OAT, we train five models per backbone (seeds 42–46), report the mean, and derive the responsible agent from the predicted step. In the with-GT setting, the gold answer is appended to the content of the first step, since StepFinder encodes no separate task description; the trained models are shared between the two settings. Finally, the regenerated training corpus shares tasks — though not trajectories — with parts of our test data: 22 of 85 TE-Cap, 51 of 91 TE-Mag, and 30 of 50 CE-GAIA trajectories are built on tasks that also appear in training, while both Who&When subsets are disjoint. This overlap can only favor the baseline, so we leave it in place.

**Evaluation.** Both baselines write one prediction per trajectory; a trajectory that receives no usable prediction — for OAT, one whose annotated step is never scored — counts as an error. A reported cell is the mean over the three test splits of the subset’s seed triple and, within each split, over the five training seeds, under the same scoring rules as every other method in Table 1.

Table 14: Cost comparison. CRR requires one forward pass per step and no training; LLM-as-judge costs are token-dominated. Numbers per the hand-crafted subset.

| Method                     | Training | Tokens / traj. | Time / traj. (s) |
|----------------------------|----------|----------------|------------------|
| All-at-Once                | none     | –              | –                |
| Step-by-Step               | none     | –              | –                |
| Binary Search              | none     | –              | –                |
| Fine-tuned attributor      | –        | –              | –                |
| CRR (extraction + rescore) | none     | –              | –                |

Table 15: Combining CRR with an LLM judge (e.g., restrict the judge to CRR’s top- $k$  candidates). Step-Level Accuracy and token cost, hand-crafted.

| Method                              | Step-Level Acc | Tokens / traj. |
|-------------------------------------|----------------|----------------|
| Step-by-Step (alone)                | –              | –              |
| CRR (alone)                         | –              | –              |
| CRR $\rightarrow$ judge (top- $k$ ) | –              | –              |

Table 16: Robustness of CRR to the source of the SVD reference matrix  $R$  (different fitting splits / pooled across subsets). Step-Level Accuracy, hand-crafted.

| Reference source      | Step-Level Acc |
|-----------------------|----------------|
| In-subset train split | –              |
| Pooled across subsets | –              |
| Held-out trajectories | –              |

Table 17: Per-model Step-Level Accuracy (without ground truth), SVD base scorer vs. CRR. Final model identifiers to be confirmed.

| Proxy model                       | Algorithm-Generated |      | Hand-Crafted |      |
|-----------------------------------|---------------------|------|--------------|------|
|                                   | Base                | +CRR | Base         | +CRR |
| Qwen3-8B                          | –                   | –    | –            | –    |
| Llama-3.1-8B-Instruct             | –                   | –    | –            | –    |
| DeepSeek-R1-Distill-Llama-8B      | –                   | –    | –            | –    |
| Qwen3-14B                         | –                   | –    | –            | –    |
| Qwen-family (larger, $\sim 9B$ )  | –                   | –    | –            | –    |
| Qwen-family (larger, $\sim 27B$ ) | –                   | –    | –            | –    |